

Instituto Tecnológico de San Luis Potosí

Ingeniería en Sistemas Computacionales

**Lenguajes de Interfaz
11:00-12:00
Tareas Unidad 4**

Alumna:

Venegas Ordaz Fátima Vianney

Lugar y Fecha:
Soledad de Graciano Sánchez, San Luis Potosí, 11 de abril de 2026

Índice

Introducción.....	3
Programación para dispositivos de entrada (Scanner y teclado).....	4
Teclado.....	4
Scanner.....	6
Programación de dispositivos de salida (Impresoras y Cámaras de video).....	9
Impresoras.....	9
Cámaras de video.....	11
Herramientas para la programación de dispositivos y su código.....	13
STM32CubeIDE.....	13
SparkFun RedBoard Artemis.....	14
MPLAB [®] X Entorno de desarrollo integrado (IDE).....	16
Conclusión.....	18
Referencias.....	19

Introducción.

El presente documento tiene como objetivo hablar sobre la relación técnica entre el hardware y el software a través de la programación de diversos dispositivos de entrada y salida, así como las herramientas que existen como alternativa a Arduino para la programación de dichos dispositivos. Se explora cómo a través del código se traducen señales físicas e impulsos eléctricos en datos digitales procesables, utilizando protocolos de comunicación y APIs específicas para garantizar la interconectividad y funcionalidad del sistema de los dispositivos. Si bien Arduino es el estándar para prototipado rápido, el entorno industrial y profesional exige herramientas con mayor control sobre los recursos del hardware.

Programación para dispositivos de entrada (Scanner y teclado).

Teclado.

Un teclado es un dispositivo de entrada que te permite escribir letras, números y símbolos en tu laptop, PC u otro dispositivo electrónico.

Cuando presionas una tecla en el teclado, envía una señal eléctrica al procesador de tu computadora, que luego interpreta esa señal y muestra la letra o símbolo correspondiente en la pantalla.

En Java, se utiliza la interfaz `KeyListener`. Esta permite capturar el momento exacto en que una tecla es presionada, liberada o escrita. (Lenovo México, s.f.)

Lógica de programación: El hardware del teclado envía un "scancode" (código de rastreo) al sistema operativo cada vez que se cierra el circuito de una tecla. Java traduce esto en un `KeyEvent`, permitiendo al programador identificar la tecla mediante su código virtual (`VK_CODE`). (Oracle, 2024)

Código de implementación.

```
import java.awt.event.*; // Importamos las herramientas de eventos
```

```
import javax.swing.*; // Importamos para la ventana gráfica
```

```
public class ControlTeclado extends JFrame {
```

```
    public ControlTeclado() {
```

```
        // Configuramos la ventana básica
```

```
        setTitle("Captura de Hardware: Teclado");
```

```
        setSize(400, 200);
```

```
        setDefaultCloseOperation(JFrame.EXIT_VALUE);
```

```

// Agregamos el KeyListener: el "oído" del programa para el teclado
this.addKeyListener(new KeyListener() {

    @Override

    public void keyPressed(KeyEvent e) {

        // PASO 1: El hardware envía un código numérico (ScanCode)

        // PASO 2: Java lo traduce a un Virtual Key Code (VK)

        int codigo = e.getKeyCode();

        // PASO 3: Identificamos qué tecla física se activó

        System.out.println("Se detectó una señal eléctrica de la tecla: " +
codigo);

        // Ejemplo de lógica: si la tecla es el ENTER

        if (codigo == KeyEvent.VK_ENTER) {

            System.out.println("Acción ejecutada: El usuario confirmó con
ENTER.");

            }

        }

    }

// Estos métodos son obligatorios por la interfaz, aunque no se usen

@Override

```

```

public void keyReleased(KeyEvent e) {

    // Se activa cuando el usuario deja de presionar la tecla

}

@Override

public void keyTyped(KeyEvent e) {

    // Se activa cuando la tecla genera un carácter (letras/números)

}

});

}

}

```

Scanner.

Un escáner es un dispositivo que digitaliza documentos, imágenes o incluso objetos, convirtiéndolos en archivos digitales que puedes visualizar, editar y almacenar en tu computadora.

Para interactuar con un escáner desde Python en entornos como Windows, se utiliza comúnmente la librería pywin32 para comunicarse con la API WIA (Windows Image Acquisition). (Microsoft Learn, 2018)

Lógica de programación: A diferencia de un teclado, el escáner no envía datos constantes. El programa debe enviar una instrucción de "escaneo" al dispositivo. La API WIA gestiona la comunicación con el controlador (driver), activa la lámpara y el sensor del escáner, y devuelve los datos en forma de un flujo de bits que se guarda como imagen. (Hammond, 2023)

Código de implementación.

```
import win32com.client # Librería para hablar con el sistema operativo
Windows
```

```
def ejecutar_escaneo():
    try:
        # PASO 1: Creamos un objeto para manejar dispositivos de imagen
(WIA)
        # Esto es como abrir la puerta a los controladores del escáner
        manager = win32com.client.Dispatch("WIA.DeviceManager")

        # PASO 2: Seleccionamos el hardware. manager.DeviceInfos(1)
        # apunta al primer escáner físico conectado al puerto USB.
        dispositivo = manager.DeviceInfos(1).Connect()

        print("Hardware conectado exitosamente. Iniciando lámpara y sensor...")

        # PASO 3: Accedemos al 'Item' (el lente/sensor del escáner)
        # y ordenamos la transferencia de datos (el escaneo real)
        item_escaneo = dispositivo.Items(1)
        imagen_digital = item_escaneo.Transfer()

        # PASO 4: Los datos binarios recibidos se guardan como archivo .jpg
        ruta_archivo = "resultado_escaneo.jpg"
```

```
imagen_digital.SaveFile(ruta_archivo)
```

```
print(f"El proceso terminó. Archivo generado: {ruta_archivo}")
```

```
except Exception as e:
```

```
    # Si el escáner está apagado o no hay hardware, capturamos el error
```

```
    print(f"Error de hardware: {e}")
```

```
if __name__ == "__main__":
```

```
    ejecutar_escaneo()
```

Programación de dispositivos de salida (Impresoras y Cámaras de video)

Impresoras.

Una impresora es un dispositivo de salida gestionado por un gestor de impresión de servidor y diseñado para imprimir documentos en papel.

La impresora es un periférico de salida física. Su programación requiere interactuar con el "Gestor de Impresión" del sistema operativo para enviar datos que el hardware pueda convertir en puntos de tinta o tóner. (IBM, s.f.)

Lógica de programación: El software no habla directamente con los motores de la impresora, sino que utiliza una API (como javax.print) para crear un Trabajo de Impresión (Print Job). Este trabajo envía una señal electrónica que la placa de control de la impresora traduce en coordenadas para el cabezal de impresión. (Oracle, 2024)

Código de implementación.

```
import javax.print.*; // Librería para servicios de impresión

import java.io.*;

public class Milmpresora {

    public static void main(String[] args) {

        // Primero, le pedimos a Java que busque la impresora que tenemos de
        // fábrica

        PrintService milmpresora =
        PrintServiceLookup.lookupDefaultPrintService();

        if (milmpresora != null) {

            System.out.println("Encontré la impresora: " +
            milmpresora.getName());
```

```

// Aquí definimos que lo que vamos a mandar es texto sencillo
DocFlavor tipoTexto = DocFlavor.INPUT_STREAM.AUTOSENSE;

// Creamos el "trabajo", que es como la orden para que la impresora
empiece
DocPrintJob tarea = milmpresora.createPrintJob();

// Escribimos lo que queremos que salga en el papel
String texto = "Hola Profe, esta es una prueba de salida desde Java.";

InputStream          flujoDatos          =          new
ByteArrayInputStream(texto.getBytes());

Doc miDoc = new SimpleDoc(flujoDatos, tipoTexto, null);

try {
    // Aquí es donde el programa manda la señal electrónica a la
    impresora

    tarea.print(miDoc, null);

    System.out.println("¡Listo! El texto se envió a la impresora.");
} catch (PrintException e) {
    System.out.println("Hubo un error al conectar con el hardware.");
    }
    }
}

```

Cámaras de video.

Una videocámara o cámara de video es un dispositivo, generalmente portátil, que permite registrar imágenes y sonidos. Convirtiéndolos en señales eléctricas que pueden ser reproducidos por un aparato determinado. En otras palabras, una cámara de vídeo es un traductor óptico.

Su programación consiste en capturar la señal eléctrica que genera la luz al golpear el sensor y mostrarla como una salida digital en tiempo real. (Euroinnova, 2024)

Lógica de programación: El programa utiliza la librería OpenCV para gestionar el hardware. La lente captura fotones que el sensor CMOS convierte en señales eléctricas. El software "lee" estas señales miles de veces por segundo y las traduce en píxeles que forman el video en la pantalla. (OpenCV, 2024)

Código de implementación.

```
import cv2 # Usamos esta librería que es la mejor para manejar cámaras

def usar_camara():

    # Conectamos con la cámara (el 0 significa que es la primera que
    encuentre)

    # Esto le dice al hardware que se encienda

    mi_video = cv2.VideoCapture(0)

    print("Prendiendo la cámara... Presiona 'q' para salir.")

    # Como el video no se detiene, usamos un ciclo 'while' para capturar todo
```

```

while True:

    # Aquí le pedimos a la cámara que nos dé la imagen que está viendo
    ahorita

    ok, cuadro = mi_video.read()

    if not ok:

        print("No se pudo leer la señal de la cámara.")

        break

    # Esta línea abre una ventanita para mostrarnos el video en la pantalla

    cv2.imshow('Mi Video en Vivo', cuadro)

    # Si el alumno (o usuario) presiona la tecla 'q', el video se detiene

    if cv2.waitKey(1) & 0xFF == ord('q'):

        break

    # Muy importante: al final hay que "soltar" la cámara para que se apague
    el foquito

    mi_video.release()

    cv2.destroyAllWindows()

    print("Cámara apagada correctamente.")

if __name__ == "__main__":

    usar_camara()

```

Herramientas para la programación de dispositivos y su código.

STM32CubeIDE.

STM32CubeIDE es un entorno para editar, compilar y depurar código. Los usuarios suelen utilizarlo junto con otras herramientas del ecosistema STM32Cube, como la herramienta de configuración gráfica STM32CubeMX. También es compatible con diversas herramientas de soluciones verticales, como TouchGFXDesigner, STM32Cube AI Studio y Motor Control Workbench.

Utiliza una arquitectura de capas. Tú programas sobre una capa llamada HAL (Hardware Abstraction Layer), que traduce tus comandos a lenguaje de registros que el chip entiende. (STMicroelectronics, 2024)

Código demostración en C.

```
#include "main.h"

// Función principal del sistema

int main(void) {

    // 1. Inicialización de la capa de abstracción de hardware

    HAL_Init();

    // 2. Configuración de los pines (GPIO)

    // En STM32, los periféricos están apagados por defecto para ahorrar
    energía

    __HAL_RCC_GPIOA_CLK_ENABLE(); // Habilitar reloj del puerto A

    GPIO_InitTypeDef GPIO_InitStruct = {0};

    GPIO_InitStruct.Pin = GPIO_PIN_5;
```

```

GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP; // Salida Push-Pull

GPIO_InitStruct.Pull = GPIO_NOPULL;

GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;

HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);


// 3. Bucle infinito de control del dispositivo

while (1) {

    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5); // Alternar estado físico del
pin

    HAL_Delay(500); // Retraso basado en el SysTick del procesador

}

} (STMicroelectronics, 2024)

```

SparkFun RedBoard Artemis.

El SparkFun Artemis es un módulo increíble. ¡Una gran cantidad de funcionalidades en un espacio reducido de 10x15 mm! Pero lo que realmente lo hace potente es la capacidad de escribir bocetos rápidamente y crear proyectos usando solo código Arduino. (SparkFun Electronics, s.f.)

A diferencia de los procesadores básicos, el Artemis utiliza un núcleo ARM Cortex-M4F con una velocidad de hasta 96MHz. Lo que lo hace especial para la programación de dispositivos es su capacidad de "morder" ráfagas de datos de sensores y procesarlas con IA sin necesidad de conectarse a la nube.

Se programa principalmente en C y C++. También soporta TensorFlow Lite, permitiendo cargar modelos de inteligencia artificial (como reconocimiento de voz o gestos) directamente en el chip.

Este código muestra cómo se inicializa el hardware para trabajar con TensorFlow Lite Micro, algo imposible en un Arduino convencional. (Torrente, 2023)

Código en c++.

```
#include "tensorflow/lite/micro/all_ops_resolver.h"

#include "tensorflow/lite/micro/micro_interpreter.h"

#include "tensorflow/lite/schema/schema_generated.h"


// Definición del área de memoria para el modelo de IA (Tensor Arena)

const int kTensorArenaSize = 10 * 1024;

uint8_t tensor_arena[kTensorArenaSize];


void setup_artemis_ia() {

    // 1. Cargar el modelo desde la memoria Flash del dispositivo

    const tflite::Model* model = tflite::GetModel(g_model_data);


    // 2. Configurar el intérprete para ejecutar el modelo en el núcleo Cortex-

static tflite::MicroInterpreter interpreter(

    model, resolver, tensor_arena, kTensorArenaSize, error_reporter);


    // 3. Asignar memoria física del hardware a los tensores

    interpreter.AllocateTensors();

}
```

M4

```

void loop_ia() {

    // Ejecutar inferencia de IA localmente en el dispositivo

    interpreter.Invoke();

}

```

MPLAB® X Entorno de desarrollo integrado (IDE)

MPLAB X Integrated Development Environment (IDE) es un programa de software ampliable y altamente configurable que incorpora potentes herramientas para ayudarle a descubrir, configurar, desarrollar, depurar y validar diseños embebidos para la mayoría de nuestros microcontroladores y controladores de señales digitales. MPLAB X IDE funciona a la perfección con el ecosistema de software y herramientas de desarrollo de MPLAB, muchas de las cuales son totalmente gratuitas.

Se utiliza un flujo de **Compilación y Depuración (Debug)**. El programador debe conocer el mapa de memoria del dispositivo.

En esta herramienta, el programador debe configurar los "Configuration Bits" (fusibles del hardware) antes de iniciar, lo cual define cómo se comportará el oscilador físico y los sistemas de protección del chip. (Microchip Technology, 2024)

Código de demostración en C.

```

// Configuración de bits (Fusibles del hardware)

#pragma config FOSC = INTOSCIO // Oscilador interno

#pragma config WDTE = OFF      // Perro guardián desactivado (Watchdog
Timer)

#pragma config PWRTE = ON      // Temporizador de encendido activado

#include <xc.h>                // Biblioteca principal del compilador XC8

```

```
#define _XTAL_FREQ 4000000    // Definir frecuencia de 4MHz para  
retardos
```

```
void main(void) {
```

```
    // 1. Configuración de registros de dirección (TRIS)
```

```
    // TRISB = 0 significa que todo el Puerto B es SALIDA
```

```
    TRISB = 0x00;
```

```
    // 2. Inicializar el estado de los pines
```

```
    (LAT) LATB = 0x00;
```

```
    while(1) {
```

```
        // 3. Manipulación directa de los niveles lógicos del
```

```
        hardware LATBbits.LATB0 = 1; // Poner el pin RB0 en
```

```
        estado alto (5V)
```

```
        __delay_ms(500);    // Retraso de medio segundo
```

```
        LATBbits.LATB0 = 0; // Poner el pin RB0 en estado bajo (0V)
```

```
        __delay_ms(500);
```

```
    }
```

```
}
```

Conclusión.

Como conclusión tenemos que la programación de dispositivos de entrada y de salida demuestran que cada dispositivo requiere un tratamiento lógico distinto según su naturaleza o sus requerimientos, desde la captura de eventos asíncronos en el teclado hasta la gestión de flujos de datos bajo demanda en el escáner. Además de que es fundamental conocer su funcionamiento ya que nos permite comprender como se comunica la parte física con la parte lógica.

También como parte es fundamental saber que el elegir una herramienta de programación depende directamente de la complejidad del dispositivo y los requisitos de eficiencia del proyecto. El dominio de estas herramientas profesionales es fundamental para cualquier ingeniero que busque diseñar soluciones tecnológicas.

Referencias.

Euroinnova Business School. (2024). ¿Cómo funciona una cámara de vídeo?. Recuperado de <https://www.euroinnova.com/blog/como-functiona-una-camara-de-video>

IBM. (s.f.). Conceptos de impresora: Gestión de impresión y servidores. Recuperado de <https://www.ibm.com/docs/es/cmofm/9.5.0?topic=concepts-printer>

Hammond, M. (2023). Python for Windows (pywin32) Extensions.
Recuperado de <https://github.com/mhammond/pywin32>

Lenovo México. (s.f.). ¿Qué es el teclado de una computadora? Glosario de tecnología. Recuperado de <https://www.lenovo.com/mx/es/glosario/teclado/>

Microchip Technology. (2024). MPLAB® X Integrated Development Environment (IDE). Recuperado de <https://www.microchip.com/en-us/tools-resources/develop/mplab-x-ide>

Microsoft Learn. (2018, 31 de mayo). Windows Image Acquisition (WIA) - Win32 apps. Recuperado de <https://learn.microsoft.com/en-us/previous-versions/windows/desktop/wia/-wia-startpage>

OpenCV. (2024). Getting Started with Video: Capture, Display, and Save.
Recuperado de https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html

Oracle. (2024). Java Print Service User Guide. Recuperado de <https://docs.oracle.com/javase/8/docs/technotes/guides/jps/index.html>

Oracle. (2024). KeyEvent (Java Platform SE 8). Recuperado de <https://docs.oracle.com/javase/8/docs/api/java/awt/event/KeyEvent.html>

SparkFun Electronics. (s.f.). Artemis Development with Arduino: All tutorials. Recuperado de <https://learn.sparkfun.com/tutorials/artemis-development-with-arduino/all>

STMicroelectronics. (2024). STM32CubeIDE: Integrated Development Environment for STM32. Recuperado de <https://www.st.com/en/development-tools/stm32cubeide.html>

Torrente, A. (2023). Alternativas a Arduino: Microcontroladores de alto rendimiento y bajo consumo. Recuperado de <https://descubrearduino.com/alternativas-arduino/>